

ESP32 cheat sheet

All the key numbers, pins and commands from the course on a few pages. Keep it next to your workbench.

1. Pins (DevKitC / DevKit V1)

Use	Suggested pins	Note
This course's fixed pins	LED 4 · button 27 (2nd 33) · pot 34 · LDR 35 · DHT22 13 · buzzer 25 · servo/NeoPixel 32 · relay/motor 26 (2nd 14)	The pin plan from lesson 1.3; I2C, SPI and UART2 as in the rows below
General output (LED, relay)	4, 13, 16–19, 21–23, 25–27, 32, 33	At most 20 mA per pin (absolute limit about 40)
Analog input with Wi-Fi on	32, 33, 34, 35, 36, 39 (ADC1)	ADC2 doesn't work while Wi-Fi is on
Input only	34, 35, 36, 39	No internal pull-up
I2C	SDA 21 · SCL 22	Change them with <code>Wire.begin(sda, scl)</code>
SPI (VSPI)	MOSI 23 · MISO 19 · SCK 18 · CS 5	A separate CS for each device
UART2	RX2 16 · TX2 17	Always write the pins explicitly: <code>Serial2.begin(9600, SERIAL_8N1, 16, 17)</code> ; without them, core 3 uses GPIO4 and GPIO25. Module TX goes to board RX, and vice versa.
DAC	25, 26	8-bit; not on the S3, C3 or C6
Touch	T0=4 T1=0 T2=2 T3=15 T4=13 T5=12 T6=14 T7=27 T8=33 T9=32	<code>touchRead(pin)</code>
Strapping (careful)	0, 2, 5, 12, 15	Nothing should pull them high or low at power-up
Forbidden	6–11	Internal flash
Waking from deep sleep	0, 2, 4, 12–15, 25–27, 32–39	RTC GPIOs only

2. Key numbers

Item	Value
Logic voltage	3.3 V (the pins are not 5 V tolerant)
ADC	12-bit: 0 to 4095
Standard serial speed	115200 baud
Processor	Dual-core Xtensa LX6 up to 240 MHz, about 520 KB of SRAM
Wi-Fi	2.4 GHz only
Deep sleep (chip alone)	About 10 μ A; a dev board draws a few mA because of its regulator and LED
ESP-NOW	At most 250 bytes per message (v1); v2 allows up to 1470 bytes
SG90 servo	50 Hz · 0.5–2.4 ms
DHT22	At least 2 seconds between two readings
SSD1306 OLED	I2C address usually 0x3C

3. Most-used commands (Arduino-ESP32 core 3)

cheatsheet.ino

Copy code

```
1 pinMode(pin, OUTPUT);           // INPUT, INPUT_PULLUP, INPUT_PULLDOWN
2 digitalWrite(pin, HIGH);        // HIGH = 3.3V, LOW = 0V
3 int b = digitalRead(pin);       // 0 or 1
4 int a = analogRead(pin);        // 0..4095
5 int mv = analogReadMilliVolts(pin);
6 ledcAttach(pin, 5000, 8);       // PWM: frequency and resolution bits
7 ledcWrite(pin, 128);            // 0..255 at 8 bits
8 unsigned long t = millis();     // ms since power-up
9 Serial.begin(115200); Serial.println(x);
10 Wire.begin(21, 22);            // I2C
11 WiFi.begin(ssid, pass); WiFi.status() == WL_CONNECTED;
12 esp_sleep_enable_timer_wakeup(10ULL * 1000000); esp_deep_sleep_start();
```

L1-3 Digital. Set a pin's role, write and read ones and zeros.

Why this way? The most basic thing you do with any GPIO; lessons 2.2, 2.3 and 3.1.

L4-5 Analog. The raw ADC number, or the voltage in millivolts.

Why this way? The millivolt version is calibrated and easier for a beginner to understand; lesson 3.1.

L6-7 PWM in core 3. In core 2 this was `ledcSetup` and `ledcAttachPin`.

Why this way? Older tutorials online are written for core 2 and don't compile on core 3; lesson 3.2.

L8 Timing without stopping.

Why this way? The replacement for delay when you need to do several things at once; lesson 3.1.

L9-11 Communication. Serial, I2C and Wi-Fi.

Why this way? Lessons 2.2, 3.3 and 4.1.

L12 A 10-second deep sleep. The time is in microseconds and must be a 64-bit number (`ULL`).

Why this way? Without ULL the big number overflows; lesson 6.1.

4. Formulas

Formula	Example
Ohm's law: $V = I \times R$	LED resistor: $(3.3 - 2.0) \div 0.006 \approx 220\Omega$
Voltage divider: $V_{out} = V_{in} \times R2 \div (R1 + R2)$	$3.3 \times 10k \div 20k = 1.65V$
Voltage from the ADC: $V \approx raw \times 3.3 \div 4095$	2048 $\rightarrow$ about 1.65V (approximate; better: <code>analogReadMilliVolts</code>)
Battery life: $hours = mAh \div average\ current\ (mA)$	$2000 \div 0.5mA = 4000\ hours$ (less in practice)
PWM: $duty = fraction \times (2^{bits} - 1)$	50% at 8 bits: $0.5 \times 255 \approx 128$

5. Wire colors

● Red: power · ● Black: ground · ● Orange: output · ● Blue: input and data · ● Green: clock (SCL and SCK) · ● Yellow: SDA and TX

6. ESP32 models at a glance

Model	Processor	Bluetooth Classic	DAC	Touch	Built-in USB	Strapping pins
ESP32	2× Xtensa LX6	Yes	25, 26	Yes	No	0, 2, 5, 12, 15
ESP32-S2	1× Xtensa LX7	No Bluetooth at all	Yes	Yes	OTG	0, 45, 46
ESP32-S3	2× Xtensa LX7	BLE only	No	Yes	OTG + Serial/JTAG	0, 3, 45, 46
ESP32-C3	1× RISC-V	BLE only	No	No	Serial/JTAG	2, 8, 9
ESP32-C6	1× RISC-V (Wi-Fi 6)	BLE only	No	No	Serial/JTAG	4, 5, 8, 9, 15

On the C3 and S3, if the Serial Monitor shows nothing: `Tools > USB CDC On Boot > Enabled` .
Details in [lesson 6.4](#).

7. Heavy loads and real-project tools

Item	Rule
NPN transistor base resistor	$R_b = (3.3 - 0.7) \div I_b$ and $I_b \geq I_c \div hFE \times 2.5$ (usually 470 Ω to 1 k Ω)
MOSFET	Logic-level only (A03400, IRLZ44N); 100 Ω gate resistor, 10 k Ω pull-down
Coil loads (relay, motor)	1N4007 flyback diode in parallel, reversed
WS2812 current	Up to about 60 mA per LED at full white: 30 LEDs $\approx$ 1.8 A
FreeRTOS task	<code>xTaskCreatePinnedToCore(fn, "name", 4096, NULL, 1, NULL, 1);</code> (stack size in bytes)
OTA	Needs a partition scheme with two app slots; Huge APP has no over-the-air updates

Full explanation: [lesson 3.4](#), [lesson 5.4](#) (FreeRTOS) and [lesson 5.5](#) (OTA).

8. Ten common errors

Symptom	Check this first
The port doesn't show up	Data cable, CP2102 or CH340 driver
<code>Failed to connect ... packet header</code>	Hold BOOT while it says Connecting
Garbled characters in the Serial Monitor	Speed 115200
<code>Brownout detector was triggered</code>	Weak cable or power supply, especially with Wi-Fi or a servo
The board won't boot after you connect a part	Strapping pins (0, 2, 12, 15)
analogRead always 0 or 4095 with Wi-Fi on	It's an ADC2 pin; move to ADC1
The LED won't light	LED direction, resistor, common GND
One button press counts several times	Debounce
Wi-Fi won't connect	5 GHz network, password, distance
<code>'ledcSetup' was not declared</code>	Core 2 code on core 3; use ledcAttach